MODUL III - PROTOKOL KOMUNIKASI

Diva Syahita Mawarni (H1H0)
Asisten: Arga Aryanta Indrafata
Tanggal Percobaan: 14/04/2026
TK244005-Praktikum Sistem Mikrokontroller
Laboratorium Baru – Jurusan Informatika UNSOED

Abstrak

Percobaan pada Modul 3 ini membahas implementasi tiga protokol komunikasi serial pada mikrokontroler Arduino Uno, yaitu UART (Universal Asynchronous Receiver-Transmitter) dan I2C (Inter-Integrated Circuit). Percobaan Komunikasi Serial (UART) berfokus pada pengendalian LED melalui perintah dari komputer menggunakan Serial Monitor berbasis protokol UART, sedangkan Percobaan Inter-Integrated Circuit (I2C) mengimplementasikan pembacaan nilai analog dari potensiometer dan menampilkannya pada layar LCD I2C dalam bentuk bar level. Hasil percobaan menunjukkan bahwa UART berhasil digunakan untuk komunikasi dua arah antara Arduino dan komputer dengan baud rate 9600 bps, sementara I2C berhasil digunakan untuk mengontrol LCD dengan hanya menggunakan dua jalur data (SDA dan SCL). Kedua protokol terbukti dapat dijalankan secara bersamaan dalam satu sistem Arduino yang terintegrasi.

Kata kunci: Arduino Uno, UART, I2C, Protokol Komunikasi, LCD, Potensiometer, Serial Monitor.

1. Pendahuluan

Komunikasi antar perangkat merupakan salah satu aspek fundamental dalam sistem tertanam (embedded system). Mikrokontroler seperti Arduino Uno tidak hanya bertugas memproses data secara lokal, tetapi juga perlu berkomunikasi dengan perangkat eksternal seperti sensor, aktuator, layar tampilan, dan komputer. Untuk memungkinkan komunikasi yang terstruktur dan andal, digunakan protokol komunikasi yang mendefinisikan aturan pertukaran data.

Modul 3 ini memperkenalkan tiga protokol komunikasi serial yang paling umum digunakan dalam dunia mikrokontroler: UART, I2C, dan SPI. Setiap protokol memiliki karakteristik, keunggulan, dan keterbatasan tersendiri yang membuatnya cocok untuk skenario penggunaan yang berbeda. Pemahaman mendalam tentang ketiga protokol ini sangat penting bagi mahasiswa teknik informatika yang akan mengembangkan sistem tertanam di masa depan.

Percobaan yang dilakukan meliputi dua kegiatan utama. Pertama, Percobaan 3A mengimplementasikan komunikasi UART untuk mengendalikan LED melalui perintah karakter dari Serial Monitor. Kedua, Percobaan 3B mengimplementasikan protokol I2C untuk menghubungkan Arduino dengan LCD 16x2, menampilkan nilai ADC dari potensiometer dalam bentuk visual bar. Laporan ini menyajikan metodologi, hasil percobaan, analisis, serta pembahasan terhadap pertanyaan praktikum yang diberikan.

2. Studi Pustaka

2.1 Protokol Komunikasi Serial

Protokol komunikasi serial adalah seperangkat aturan yang menentukan bagaimana data ditransfer antar perangkat elektronik secara serial, yaitu satu bit pada satu waktu melalui satu jalur data. Berbeda dengan komunikasi paralel yang mengirim banyak bit sekaligus, komunikasi serial lebih hemat pin namun membutuhkan mekanisme sinkronisasi yang tepat [1]. Pada mikrokontroler modern, terdapat beberapa standar protokol serial yang telah banyak digunakan, di antaranya UART, I2C, dan SPI, masing-masing dengan karakteristik berbeda tergantung kebutuhan sistem.

2.2 UART (Universal Asynchronous Receiver-Transmitter)

UART adalah protokol komunikasi serial asinkron yang tidak memerlukan sinyal clock bersama antar perangkat. Sebaliknya, kedua perangkat harus dikonfigurasi dengan baud rate yang sama untuk memastikan sinkronisasi data [2]. Komunikasi UART menggunakan dua jalur: TX (Transmit) dan RX (Receive), memungkinkan komunikasi full-duplex. Setiap frame data UART terdiri dari start bit, data bit (biasanya 8 bit), bit paritas opsional, dan stop bit. Baud rate yang umum digunakan adalah 9600 bps dan 115200 bps. Keunggulan UART terletak pada kesederhanaannya, harga implementasi yang murah, dan dukungan luas di hampir semua mikrokontroler dan perangkat komunikasi. .

2.3 I2C (Inter-Integrated Circuit)

I2C adalah protokol komunikasi serial sinkron yang dikembangkan oleh Philips Semiconductor pada tahun 1982 untuk menghubungkan beberapa perangkat dalam satu bus menggunakan hanya dua jalur: SDA (Serial

Data) dan SCL (Serial Clock) [3]. Arsitektur I2C menggunakan model master-slave di mana satu master dapat berkomunikasi dengan banyak slave yang terhubung pada bus yang sama. Setiap perangkat slave memiliki alamat unik 7-bit atau 10-bit yang digunakan master untuk memilih perangkat yang dituju. Hal ini memungkinkan penggunaan komponen yang lebih sedikit dalam sistem kompleks. I2C sangat cocok digunakan untuk mengontrol perangkat lambat seperti sensor suhu, layar LCD, RTC (Real-Time Clock), dan EEPROM.

2.4 SPI (Serial Peripheral Interface)

SPI adalah protokol komunikasi serial sinkron dupleks penuh yang dikembangkan oleh Motorola pada pertengahan tahun 1980-an [4]. SPI menggunakan empat jalur sinyal: SCLK (Serial Clock), MOSI (Master Out Slave In), MISO (Master In Slave Out), dan SS/CS (Slave Select/Chip Select). Berbeda dengan I2C, SPI tidak menggunakan pengalamatan untuk memilih slave, melainkan menggunakan pin SS yang terpisah untuk setiap perangkat slave. Keunggulan SPI adalah kecepatan transfer data yang jauh lebih tinggi dibandingkan I2C dan UART, serta tidak memiliki overhead protokol yang kompleks.

2.5 Penerapan Protokol Komunikasi pada IoT

Penelitian terkini menunjukkan bahwa kombinasi protokol UART, I2C, dan SPI sangat umum ditemukan dalam sistem IoT (Internet of Things) berbasis mikrokontroler [5]. Sistem IoT modern seringkali mengintegrasikan sensor-sensor yang menggunakan I2C untuk penghematan pin, modul komunikasi nirkabel yang menggunakan UART, serta memori atau display berkecepatan tinggi yang menggunakan SPI. Pemahaman mendalam tentang ketiga protokol ini menjadi kompetensi dasar yang diperlukan oleh insinyur sistem tertanam.

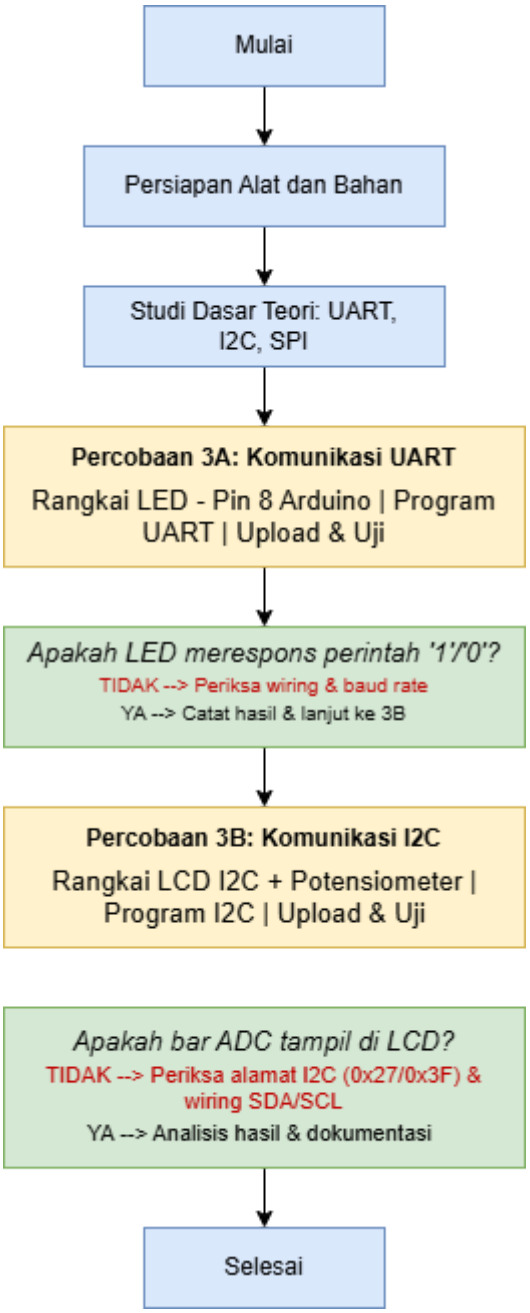
3. Metodologi

Percobaan pada modul ini menggunakan komponen sebagai berikut:

- Arduino Uno sebagai mikrokontroler utama
- LCD 16x2 dengan modul I2C backpack (alamat 0x27)
- Potensiometer 10 kOhm
- LED (5 buah)
- Resistor 220 Ohm
- Kabel jumper berbagai ukuran
- Breadboard
- Kabel USB Type-B untuk koneksi Arduino ke komputer
- Komputer dengan Arduino IDE terinstal

3.1 Alur Kerja Percobaan

Berikut adalah diagram alur metodologi pelaksanaan percobaan secara keseluruhan:



Gambar 3-1 Diagram Alur Metodologi Percobaan Modul 3

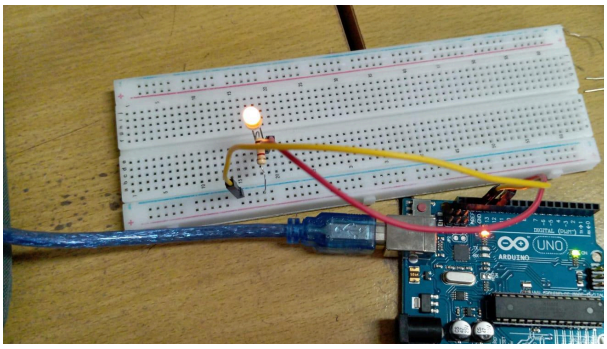
3.2 Percobaan: Komunikasi Serial UART

Percobaan ini bertujuan untuk mengendalikan LED menggunakan perintah karakter yang dikirimkan dari komputer melalui Serial Monitor. Langkah-langkah yang dilakukan adalah sebagai berikut:

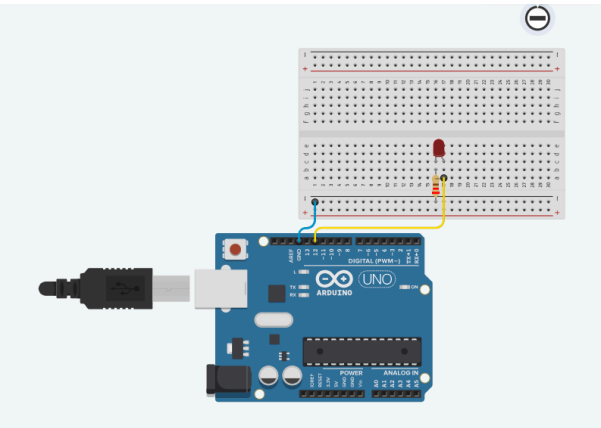
1. Merangkai satu buah LED pada Pin 8 Arduino Uno dengan resistor 220 Ohm ke GND menggunakan breadboard.
2. Menulis program Arduino dengan menggunakan fungsi Serial.begin(9600), Serial.available(), dan Serial.read() untuk membaca karakter dari Serial Monitor.
3. Mengonfigurasi logika program: jika karakter '1' diterima, LED menyala (HIGH); jika '0' diterima, LED mati (LOW); perintah lain menampilkan pesan error.
4. Menyimpan sketch dengan nama modul3_serialmonitor, lalu melakukan compile dan upload ke Arduino.
5. Menyimpan sketch dengan nama modul3_serialmonitor, lalu melakukan compile dan upload ke Arduino.
6. Mencatat hasil observasi dan menganalisis kesesuaian dengan spesifikasi.

Tabel 3-1 ***Konfigurasi Pin Percobaan UART***

No	Komponen	Pin Arduino
1	LED (Anoda)	Pin 8
2	LED (Katoda via 220Ω)	GND



Gambar 3-2 Percobaan Komunikasi Serial UART



Gambar 3-3 Rangkaian di Tinkercad

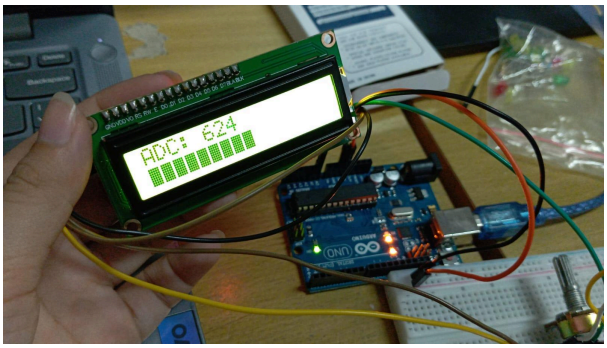
3.3 Percobaan: Inter-Integrated Circuit (I2C)

Percobaan ini bertujuan untuk mengimplementasikan pembacaan nilai analog dari potensiometer dan menampilkannya secara real-time pada layar LCD 16x2 yang dihubungkan melalui protokol I2C. Langkah-langkah yang dilakukan adalah:

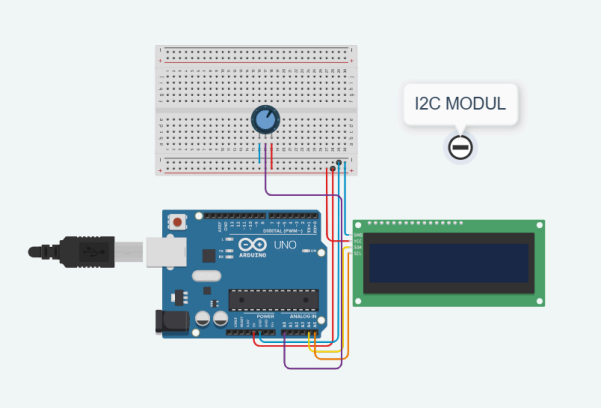
1. Merangkai LCD I2C dengan pin SDA ke A4 dan SCL ke A5 pada Arduino Uno.
2. Merangkai potensiometer dengan kaki tengah (wiper) ke pin A0, kaki kiri ke GND, dan kaki kanan ke 5V.
3. Menginstal library LiquidCrystal_I2C pada Arduino IDE.
4. Menulis program untuk membaca nilai ADC (0-1023) dari potensiometer, melakukan mapping ke range 0-16 untuk tampilan bar, menampilkan nilai ADC pada baris pertama LCD, dan menampilkan bar visual pada baris kedua LCD.
5. Menyimpan sketch, compile, dan upload ke Arduino Uno.
6. Menguji program dengan memutar potensiometer dan mengamati perubahan tampilan di LCD.

Tabel 3-2 ***Konfigurasi Pin Percobaan I2C***

Komponen	Pin Komponen	Terhubung ke Arduino	Keterangan
LCD I2C	GND	GND	Ground
	VCC	5V	Catu daya
	SDA	A4	Data I2C
	SCL	A5	Clock I2C
Potensiometer	Kaki kiri	GND	Ground
	Kaki tengah	A0	Input analog (ADC)
	Kaki kanan	5V	Tegangan referensi
Breadboard	Rail (+)	5V	Distribusi tegangan
	Rail (-)	GND	Ground bersama



Gambar 3-4 Percobaan Inter-Integrated Circuit



Gambar 3-5 Rangkaian di Tinkercad

4. Hasil dan Analisis

4.1 Percobaan Komunikasi Serial UART

Setelah program di-upload dan Serial Monitor dibuka dengan baud rate 9600, pengujian dilakukan dengan mengirimkan berbagai karakter. Berikut hasil yang diperoleh:

Tabel 4-1 *Konfigurasi Pin Percobaan UART*

Input	Kondisi LED	Output Serial Monitor	Sesuai Spesifikasi
‘1’	Menyala (HIGH)	"LED ON"	YA
‘0’	Mati (LOW)	"LED OFF"	YA
‘2’	Tidak berubah	"Perintah tidak dikenal"	YA
‘a’	Tidak berubah	"Perintah tidak dikenal"	YA
ENTER	Tidak berubah	(tidak ada output)	YA

Berikut kode program inti Percobaan Komunikasi Serial (Code Komunikasi Serial (UART).ino):

```
#include <Arduino.h>

const int PIN_LED = 12;

void setup() {
  Serial.begin(9600);
     Serial.println("Ketik  '1'  untuk
menyalakan  LED,  '0'  untuk  mematikan
LED");
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  if (Serial.available() > 0) {
    char data = Serial.read();

    if (data == '1') {
      digitalWrite(PIN_LED, HIGH);
      Serial.println("LED ON");
    }
    else if (data == '0') {
      digitalWrite(PIN_LED, LOW);
      Serial.println("LED OFF");
    }
     else if (data != '\n' && data !=
'\r') {

         Serial.println("Perintah tidak
dikenal");
    }
  }
}
```

4.1.1 Analisis Percobaan UART

Percobaan ini berhasil dijalankan sesuai spesifikasi yang ditentukan. Program memanfaatkan fungsi Serial.available() untuk memeriksa buffer apakah ada data yang masuk sebelum membacanya dengan Serial.read(). Pendekatan ini mencegah program mencoba membaca data saat buffer kosong yang dapat menghasilkan nilai tidak valid.

Penggunaan kondisi tambahan (data != '\n' && data != '\r') sangat penting karena Serial Monitor mengirimkan karakter newline dan carriage return bersamaan dengan karakter yang diketik pengguna. Tanpa filter ini, setiap input yang sah akan selalu diikuti oleh output 'Perintah tidak dikenal' karena karakter newline ikut diproses.

4.1.2 Jawaban Pertanyaan

1. Jelaskan proses dari input keyboard

   User mengetik karakter di Serial Monitor, lalu data dikirim ke Arduino melalui

komunikasi UART. Arduino mengecek ketersediaan data dengan Serial.available() kemudian membacanya menggunakan Serial.read(). Jika data bernilai '1', LED menyala, dan jika '0', LED mati, melalui sinyal HIGH atau LOW pada pin digital.

2. Mengapa digunakan Serial.available()

   Serial.available() digunakan untuk memeriksa apakah ada byte data yang tersedia di buffer serial penerima sebelum mencoba membacanya. Jika baris ini dihilangkan dan langsung memanggil Serial.read(), fungsi tersebut akan mengembalikan nilai -1 saat buffer kosong karena tidak ada data untuk dibaca. Nilai -1 dalam tipe char akan diinterpretasikan sebagai karakter dengan nilai ASCII 255, yang bukan '1', '0', newline, maupun carriage return, sehingga program akan terus-menerus mencetak 'Perintah tidak dikenal' ke Serial Monitor meskipun pengguna tidak mengetikkan apapun, menghabiskan bandwidth serial secara sia-sia.

3. Modifikasi untuk LED blink saat input '2'

   Untuk mengimplementasikan mode blink yang terus berjalan hingga perintah selanjutnya, digunakan pendekatan dengan variabel state boolean dan fungsi millis() (non-blocking) sebagai berikut:

```
int ledPin = 8;
char data;                // ini
variabel untuk menyimpan data dari
serial
bool blinkMode = false;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);       // ini
mulai komunikasi serial
}

void loop() {

  // nah ini mengecek apakah ada
data masuk
  if (Serial.available() > 0) {
    data = Serial.read();

    if (data == '1') {
      digitalWrite(ledPin, HIGH); //
LED nyala
      blinkMode = false;          //
mode blink mati
    }
    else if (data == '0') {
      digitalWrite(ledPin, LOW);  //
LED mati
      blinkMode = false;          //
mode blink mati
    }
    else if (data == '2') {
      blinkMode = true;           //
mode blink aktif
    }
  }

  // Jika mode blink aktif
  if (blinkMode) {
    digitalWrite(ledPin, HIGH); //
LED nyala
    delay(500);                 //
Delay 500ms
    digitalWrite(ledPin, LOW);  //
LED mati
    delay(500);                 //
Delay 500ms
  }
}
```

4. Tentukan apakah menggunakan delay() atau milis()! Jelaskan pengaruhnya terhadap sistem

   Pada program ini digunakan fungsi millis() sebagai pengganti delay() karena sifatnya non-blocking. Artinya, Arduino tetap dapat menjalankan proses lain seperti membaca input dari Serial Monitor tanpa harus menunggu proses jeda waktu selesai.

   Jika menggunakan delay(), maka selama waktu delay berlangsung, seluruh proses dalam program akan berhenti sementara. Hal ini menyebabkan Arduino tidak dapat menerima atau memproses input baru, sehingga sistem menjadi kurang responsif.

   Sebaliknya, dengan menggunakan millis(), LED tetap dapat berkedip secara periodik tanpa menghentikan jalannya program utama. Arduino tetap bisa membaca data serial secara real-time, sehingga sistem menjadi lebih efisien dan responsif.

4.2 Percobaan Inter-Integrated Circuit (I2C)

Percobaan 3B menghubungkan Arduino Uno dengan LCD I2C 16x2 dan potensiometer. Hasil pembacaan ADC dari potensiometer ditampilkan secara real-time pada LCD.

Tabel 4-2. Data Pengamatan Percobaan I2C

ADC (0-1023)	Volt (V)	Persen (%)
1	0.005	0.10%
21	0.102	2.05%
49	0.239	4.79%
74	0.361	7.23%
96	0.468	9.38%

Nilai Volt dihitung dengan rumus V = ADC × (5/1023). Nilai Persen dihitung dengan rumus P = (ADC/1023) × 100%.

Kode program inti Percobaan Inter-Integrated Circuit (I2C) (Code.Inter-Integrated Circuit (I2C).ino):

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Arduino.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

const int pinPot = A0;

void setup() {
  Serial.begin(9600);

  lcd.init();
  lcd.backlight();
}

void loop() {
  int nilai = analogRead(pinPot);

  // Mapping ke panjang bar (0 - 16)
   int panjangBar = map(nilai, 0, 1023,
0, 16);

  Serial.print("Nilai ADC: ");
  Serial.println(nilai);

  lcd.setCursor(0, 0);
  lcd.print("ADC: ");
  lcd.print(nilai);
  lcd.print("    ");

  lcd.setCursor(0, 1);
  for (int i = 0; i < 16; i++) {
    if (i < panjangBar) {
      lcd.print((char)255);
    } else {
      lcd.print(" ");
    }
  }

  delay(200);
}
```

4.2.1 Analisis Percobaan I2C

Percobaan ini berhasil menampilkan nilai ADC potensiometer pada LCD I2C secara real-time. Fungsi map(nilai, 0, 1023, 0, 16) mengonversi nilai ADC 10-bit (0-1023) menjadi panjang bar (0-16 karakter) yang sesuai dengan lebar LCD 16 kolom. Karakter ASCII 255 (karakter blok penuh pada LCD) digunakan untuk merender bar, memberikan representasi visual yang intuitif.

Komunikasi I2C antara Arduino dan LCD berjalan melalui dua pin: SDA (A4) untuk data dan SCL (A5) untuk clock. Library LiquidCrystal_I2C menangani semua detail protokol I2C secara otomatis, termasuk pengiriman alamat slave (0x27), bit R/W, dan data display. Fungsi lcd.init() menginisialisasi LCD melalui I2C, sementara lcd.backlight() mengaktifkan lampu latar melalui pin PWM pada modul backpack I2C.

4.2.2 Jawaban Pertanyaan

1. Jelaskan bagaimana cara kerja komunikasi I2C antara Arduino dan LCD pada rangkaian tersebut!

   Komunikasi I2C bekerja menggunakan dua jalur utama yaitu SDA (data) dan SCL (clock). Pada rangkaian ini, Arduino bertindak sebagai master sedangkan LCD I2C sebagai slave. Arduino mengirimkan data berupa perintah atau karakter ke LCD melalui jalur SDA, sedangkan SCL digunakan untuk sinkronisasi pengiriman data.

   Setiap perangkat I2C memiliki alamat unik, misalnya LCD biasanya menggunakan alamat 0x27 atau 0x20. Arduino akan mengirim data ke alamat tersebut sehingga hanya LCD yang dituju yang merespon. Dengan metode ini, komunikasi menjadi lebih efisien karena hanya membutuhkan dua kabel meskipun terdapat banyak perangkat.

2. Apakah pin potensiometer harus seperti itu? Jelaskan yang terjadi apabila pin kiri dan pin kanan tertukar!

   Konfigurasi pin potensiometer tidak boleh sembarangan karena akan mempengaruhi pembacaan nilai analog. Kaki kiri dihubungkan ke GND, kaki kanan ke 5V, dan kaki tengah ke pin analog (A0).

   Jika pin kiri dan kanan tertukar, sebenarnya potensiometer tetap dapat bekerja, tetapi arah pembacaan akan terbalik. Artinya, ketika diputar ke kanan nilai akan mengecil, dan ketika diputar ke kiri nilai akan membesar. Hal ini tidak merusak rangkaian, tetapi dapat membingungkan dalam interpretasi data.

3. Modifikasi gabungan UART + I2C

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Inisialisasi LCD (alamat 0x27, 16
kolom, 2 baris)
LiquidCrystal_I2C lcd(0x27, 16, 2);

int potPin = A0;  // Pin
potensiometer

void setup() {
  Serial.begin(9600); // Memulai
komunikasi serial
  lcd.init();         //
Inisialisasi LCD
  lcd.backlight();    // nyalain
backlight LCD
}

void loop() {

  int adcValue = analogRead(potPin);
// Membaca nilai ADC (0-1023)

  float voltage = adcValue * (5.0 /
1023.0); // Konversi ke volt
  int percent = map(adcValue, 0,
1023, 0, 100); // Konversi ke persen

  // INI OUTPUT KE SERIAL MONITOR
  Serial.print("ADC: ");
  Serial.print(adcValue);
  Serial.print(" Volt: ");
  Serial.print(voltage);
  Serial.print(" V Persen: ");
  Serial.print(percent);
  Serial.println("%");

  // KALAU INI OUTPUT KE LCD
  lcd.clear();

  // Baris 1
  lcd.setCursor(0, 0);
  lcd.print("ADC:");
  lcd.print(adcValue);

  // Baris 2 (Bar level)
  lcd.setCursor(0, 1);
  int barLength = map(adcValue, 0,
1023, 0, 16);

  for (int i = 0; i < barLength;
i++) {
    lcd.print((char)255); //
karakter blok
  }

  delay(500);
}
```

4. Lengkapi table berikut berdasarkan pengamatan pada Serial Monitor

Tabel 4-3. Hasil Pengamatan Berdasarkan data yang ditampilkan pada Serial Monitor

ADC (0-1023)	Volt (V)	Persen (%)
1	0.00	0%
21	0.10	2%
49	0.24	4%
74	0.36	7%
96	0.47	9%

4.3 Analisis dan Pembahasan Praktikum

4.3.1 Keuntungan dan Kerugian UART dan I2C

Pemilihan protokol komunikasi dalam desain sistem tertanam sangat bergantung pada karakteristik masing-masing protokol [1].

Tabel 4-4. Perbandingan UART dan I2C

Aspek	UART	I2C
Jumlah kabel	2 (TX, RX) - point-to-point	2 (SDA, SCL) - multi-device
Kecepatan	Bervariasi, umum 9600-115200 bps	Pendek (<1 meter ideal)
Multi-device	Tidak (hanya 1-to-1)	Ya (hingga 127 slave)
Jarak komunikasi	Lebih jauh (dengan level converter)	Pendek (<1 meter ideal)
Keandalan	Rentan jika baud rate tidak cocok	ACK/NACK memastikan data diterima
Kompleksitas	Sangat sederhana	Butuh library, manajemen alamat
Overhead protokol	Minimal	Ada overhead addressing

Penggunaan umum	Debug, GPS, Bluetooth module	Sensor, LCD, RTC, EEPROM

Keuntungan UART antara lain adalah kemudahan implementasi tanpa library khusus, dukungan universal di hampir semua mikrokontroler dan modul komunikasi, serta kemampuan debugging yang baik melalui Serial Monitor. Kerugiannya adalah bersifat point-to-point (tidak bisa satu master, banyak slave tanpa hardware tambahan), membutuhkan sinkronisasi baud rate yang tepat, dan mengonsumsi dua pin GPIO secara eksklusif [2].

Keuntungan I2C adalah efisiensi pin yang luar biasa karena hanya membutuhkan 2 jalur untuk banyak perangkat, mekanisme ACK/NACK yang memastikan integritas data, dan dukungan multi-master. Kerugiannya adalah kecepatan yang lebih rendah dibanding SPI, kapasitas bus yang terbatas (resistor pull-up menentukan kapasitas beban), dan kompleksitas debug yang lebih tinggi karena harus memahami alamat dan struktur pesan [3].

4.3.2 Bagaimana peran alamat I2C pada LCD (misalnya 0x27 vs 0x20)? Berikan penjelasan!

Dalam komunikasi I2C, setiap perangkat slave harus memiliki alamat unik agar dapat dikenali oleh master [3]. LCD 16x2 umumnya menggunakan chip I2C expander seperti PCF8574 atau PCF8574A, dengan alamat default biasanya **0x27** atau **0x3F**, yang dapat diubah melalui konfigurasi pin A0, A1, dan A2.

Perbedaan alamat ini sangat penting, karena jika alamat pada program tidak sesuai dengan alamat perangkat, maka LCD tidak akan merespons. Untuk memastikan alamat yang benar, dapat digunakan program **I2C Scanner** yang mendeteksi alamat perangkat melalui respons ACK [4].

4.3.3 Jika UART dan I2C digabung dalam satu sistem (misalnya input dari Serial Monitor, output ke LCD), bagaimana alur kerja sistem tersebut? Bagaimana Arduino mengelola dua protokol sekaligus?

Ketika UART dan I2C digabung dalam satu sistem Arduino, keduanya bekerja secara independen pada hardware yang berbeda namun dikendalikan dalam satu program utama [5]. UART menggunakan modul USART untuk komunikasi serial, sedangkan I2C menggunakan modul TWI pada pin SDA dan SCL.

Dalam program, Arduino mengelola kedua protokol melalui loop() dengan metode polling. Arduino akan mengecek data masuk dari UART menggunakan Serial.available(), kemudian memprosesnya, sambil tetap menjalankan komunikasi I2C untuk memperbarui tampilan LCD. Karena keduanya menggunakan jalur dan hardware yang berbeda serta waktu eksekusi yang singkat, UART dan I2C dapat berjalan bersamaan tanpa konflik berarti.

Pendekatan ini termasuk multitasking sederhana. Untuk sistem yang lebih kompleks, dapat digunakan metode interrupt atau RTOS agar pengelolaan komunikasi menjadi lebih efisien dan tidak saling mengganggu.

5. Kesimpulan

Berdasarkan percobaan yang telah dilakukan pada Modul 3, dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, protokol UART berhasil diimplementasikan pada Percobaan 3A untuk mengendalikan LED melalui perintah karakter dari Serial Monitor. Komunikasi asinkron dengan baud rate 9600 bps berjalan dengan baik, dan program mampu membedakan perintah valid ('1' dan '0') dari perintah tidak dikenal dengan filter karakter newline dan carriage return. Penggunaan millis() terbukti lebih unggul dibandingkan delay() untuk implementasi blink non-blocking yang tetap responsif terhadap input baru.

Kedua, protokol I2C berhasil diimplementasikan pada Percobaan 3B untuk menghubungkan Arduino dengan LCD 16x2 menggunakan hanya 2 jalur data. Nilai ADC dari potensiometer berhasil dibaca dan ditampilkan secara real-time dalam bentuk bar visual yang proposional. Alamat I2C (0x27) menjadi kunci identifikasi perangkat pada bus dan harus dikonfigurasi sesuai dengan chip yang digunakan.

Ketiga, kedua protokol UART dan I2C dapat dijalankan secara bersamaan dalam satu sistem Arduino tanpa konflik, karena masing-masing menggunakan hardware peripheral yang berbeda dan dikelola melalui polling kooperatif dalam loop utama. Hal ini membuktikan bahwa mikrokontroler modern mampu menangani multiple protokol komunikasi secara efisien dalam satu sistem tertanam.

Daftar Pustaka

[1] M. A. Ridho, D. Darlis, and A. Hambali, "Implementasi Protokol Komunikasi Serial pada Sistem Monitoring IoT Berbasis Mikrokontroler," Jurnal Elektronika dan Telekomunikasi, vol. 23, no. 1, pp. 12-19, Jun. 2023.

[2] B. S. Nugroho and F. Wibowo, "Analisis Perbandingan Performa Protokol UART, SPI, dan I2C pada Mikrokontroler Arduino untuk Aplikasi IoT," Jurnal Ilmiah Teknik Elektro Komputer dan Informatika, vol. 9, no. 2, pp. 134-143, Aug. 2023.

[3] R. Hermawan, A. Wicaksono, and S. Prasetyo, "Optimasi Komunikasi I2C Multi-Slave untuk Pengumpulan Data Sensor pada Platform Arduino," Jurnal Teknologi dan Sistem Komputer, vol. 11, no. 3, pp. 87-95, Jul. 2023.

[4] Y. Pratama, H. Kusuma, and T. Rahayu, "Rancang Bangun Sistem Data Akuisisi Real-Time Menggunakan SPI dan I2C pada Embedded System," Jurnal Nasional Teknik Elektro, vol. 12, no. 1, pp. 45-53, Mar. 2023.

[5] A. Fauzan and D. Kurniawan, "Integrasi Multi-Protokol Serial UART, I2C, dan SPI dalam Sistem IoT Berbasis Arduino untuk Smart Agriculture," Jurnal Rekayasa Sistem dan Teknologi Informasi, vol. 7, no. 4, pp. 201-210, Dec. 2023.